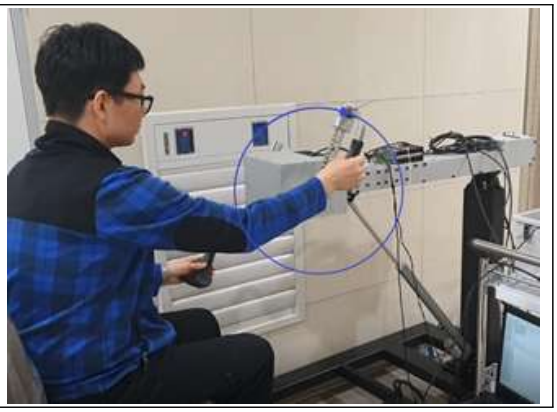
1. 운동 기능

- 운동 기능 : 2개의 운동 모드, 2개의 옵션 모드, 3개의 단계로 구성됩니다.

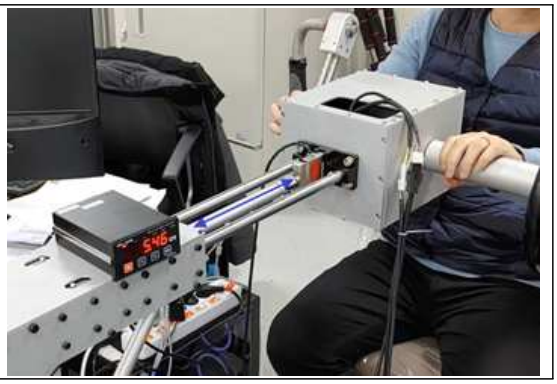
운동모드	옵션모드	단계
1-1) 원운동 (회전운동)	2-1) Active-assist	1~3
	2-2) 저항운동	
1-2) 병진운동 (전후진운동)	Active-assist	
	저항운동	

- 운동 모드

1-1) 원운동

손잡이를 잡고 회전시키는 동작을 통하여 돌리는 힘에 사용되는 근육을 풀어주거나, 근력을 강화합니다.	

1-2) 병진운동

전후진 동작을 통하여 밀고 당기는 힘에 사용되는 근육을 풀어주거나 근력을 강화합니다.	

- 옵션 모드

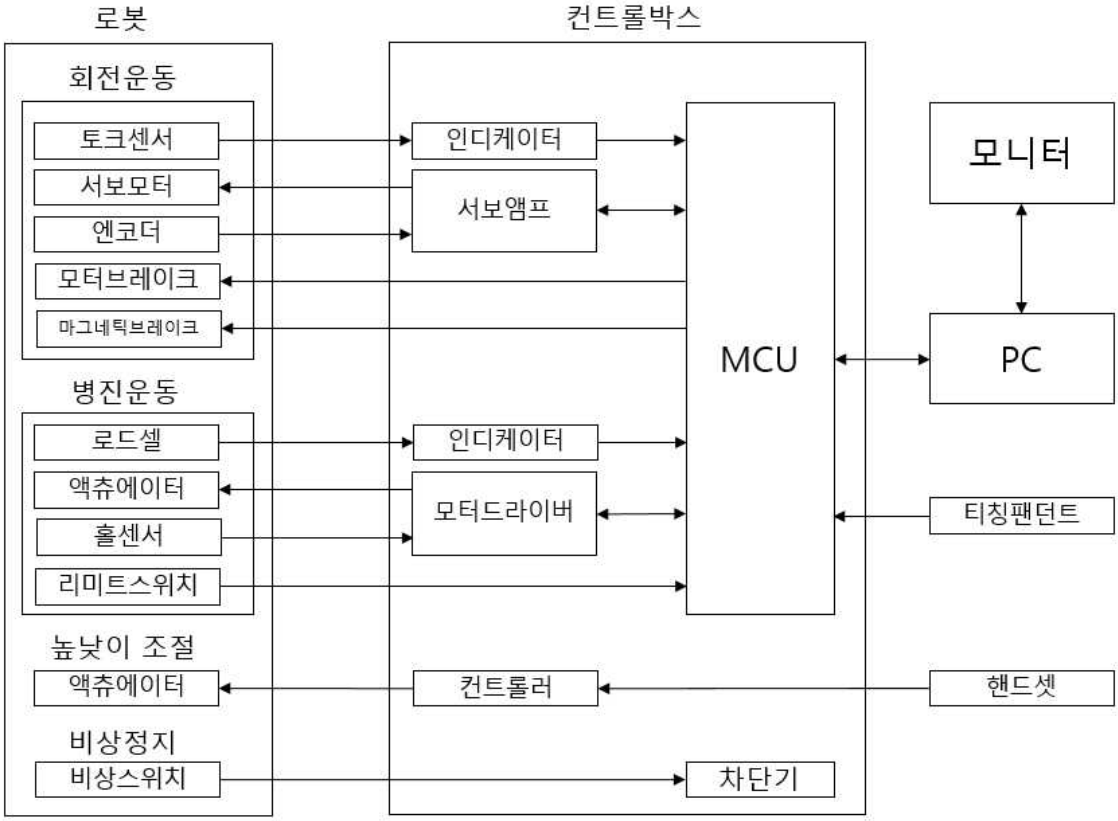
2-1) Active-assist : 모터를 작동시켜 운동을 도와줍니다. 근육을 풀어주는(스트레칭) 용도로도 사용할 수 있습니다.

2-2) 저항운동 : 근력 강화를 위하여 모터를 이용하여 물리적 저항(부하)를 생성합니다.

- 운동 단계 : 3가지 단계로 운동 강도를 조절할 수 있습니다. 높은 단계일수록 기기가 강한 힘으로 동작합니다. (1단계: 약, 2단계: 중, 3단계: 강)

2. 모듈의 구조

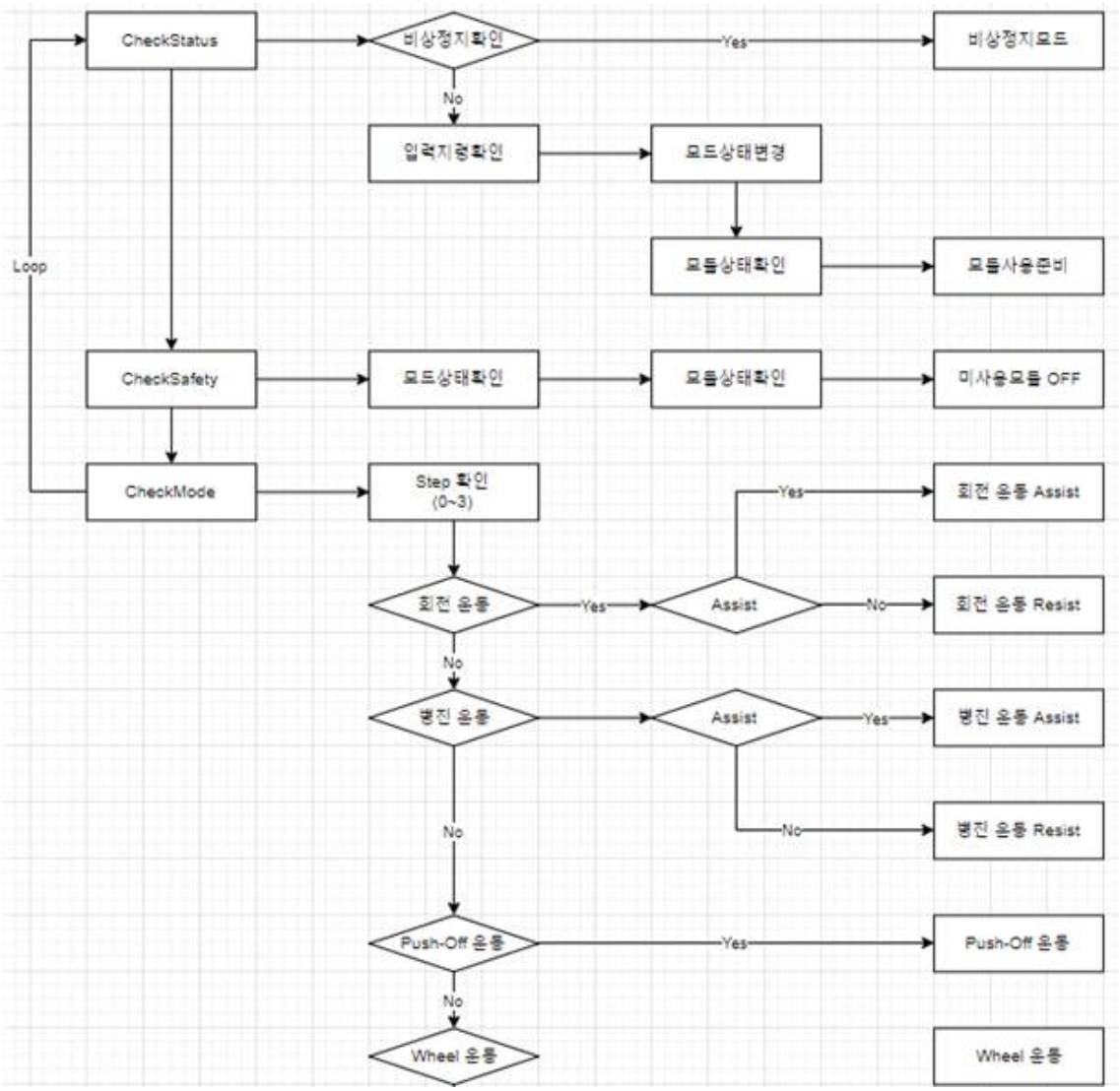
제어 모듈은 아래와 같이 구성되어 있습니다..



기능	모드	단계	구동모듈
회전 운동	Active Assist	1	AC서보앰프
		2	AC서보모터
		3	엔코더
	Resist	1	토크센서
		2	
		3	마그네틱브레이크
병진 운동	Active Assist	1	리니어액츄에이터
		2	홀센서
		3	로드셀
	Resist	1	
		2	
		3	
높낮이조절	-	-	리니어액츄에이터 핸드셋리모콘
비상정지	-	-	비상스위치

3. 메인알고리즘

메인 루프에서는 시스템 상태 확인, 안전 상태 확인, 모드 상태 확인을 반복합니다.



1) 시스템상태 확인

```
void CheckStatus(void)
{
  if(SysStatus&MASK_EMERGENCY)
    {
      Emergency();
    }
  else
    {
      HAL_UART_Receive_DMA(&huart3, (uint8_t *)rx_data, 10);
      HAL_UART_Receive_DMA(&huart4, (uint8_t *)rx_data, 10);
      ModStatus=mycommand;
    }
}
```

비상정지 상태를 가장 먼저 확인하여 비상정지 상태이면 비상정지 기능을 실행합니다.

비상정지 상태가 아니면 입력지령을 확인하여 모드상태변수를 변경합니다.
모드상태변수 : ModStatus

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
Rotate	Translate	Push-off	Wheel	Adjust Height	Assist / Resist	Step 1	Step 2
Select Mode 1					Select Mode 2	Select Step	
0 : OFF 1 : ON					0 : Resist 1 : Assist	00 : 0 01 : 1 10 : 2 11 : 3	

2) 안전상태 확인

모드 상태 변수와 모듈 상태 변수를 확인하여 모드에서 사용하지 않는 모듈의 입출력 전원 및 신호를 차단합니다.

모듈 / 모드	AC서보	브레이크	위치지령	마그네틱 브레이크	토크지령	엔코더	리니어 액츄에이터
비상정지	x	x	x	x	x	x	x
회전 Assist	o	o	o	x	o	o	x
회전 Resist	o	o	x	o	o	o	x
병진 Assist	x	x	x	x	x	x	o
병진 Resist	x	x	x	x	x	x	o
Push-off	o	o	o	o	o	o	x
Wheel	o	o	x	o	o	o	x

3) 모드상태 확인

모드상태변수(ModStatus)를 확인하여 각 운동 모드에 해당하는 함수를 호출합니다.

마스크 정의		연산결과	모드
MASK_ROTATE	0b10000000	!0	회전 운동
MASK_TRANSLATE	0b01000000	!0	병진 운동
MASK_PUSHOFF	0b00100000	!0	Push-off
MASK_WHEEL	0b00010000	!0	Wheel
MASK_HEIGHT	0b00001000	!0	높낮이 조절
MASK_ASSIST	0b00000100	!0 / 0	Assist / Resist
MASK_STEP	0b00000011	0~3	0~3 단계

4. 원운동 Active-assisted 모드

```
void Rotate_Assist(void)
{
        static uint8_t AssistTorque = 0;

        switch(Step)
        {
          case 0:
                AssistTorque = ASSIST_TORQUE_DEFAULT;
              break;
          case 1:
                AssistTorque = ASSIST_TORQUE_STEP1;
              break;
          case 2:
                AssistTorque = ASSIST_TORQUE_STEP2;
                break;
          case 3:
                AssistTorque = ASSIST_TORQUE_STEP3;
                break;
        }
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_8B_R, AssistTorque);
        EncoderCounter = TIM2->CNT;
        pulse = (int)EncoderCounter;
        if(ACservoStatus==0)
        {
                if(torque>=10)
                {
                        RotateCommand(1);
                        ACservoStatus = 1;
                }
                else
                {
                        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_1);
                        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_2);
                        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_3);
                        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_4);
                        HAL_Delay(10);
                        ACservoStatus = 0;
                }
        }
        else if(ACservoStatus==1)
        {
            if(torque<-20)
            {
                HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_1);
                HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_2);
                HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_3);
                HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_4);
                HAL_Delay(10);
                ACservoStatus = 0;
            }
            else if((torque>=-20)&&(torque<10))
            {
                RotateCommand(2);
                ACservoStatus = 1;
            }
            else if(torque>=10)
            {
                uint8_t pcmd = (uint8_t)((140+torque)/50);
                if(pcmd>9){pcmd=9;}
                RotateCommand(pcmd);
                ACservoStatus = 1;
            }
          }
}
```

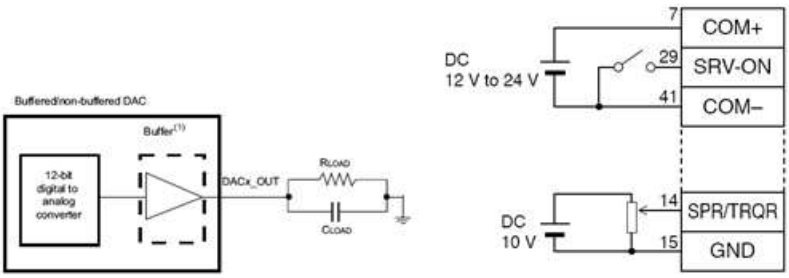
AC서보앰프로 신호 및 전압을 입력하여 AC서보모터의 위치/토크를 제어합니다. 토크 제어되고 있는 모터를 사용자가 회전시키려는 힘(토크)을 센서로 측정하여 적절한 위치로

모터를 이동합니다.

시작 시에는 정지 상태로 대기하다가 토크센서에서 일정 값 이상이 측정되면 모터를 회전합니다. 토크 값이 일정 값 이하일 경우 사용자의 팔을 당기는 힘이 커서 무리가 가는 상황으로 판단하고 정지 상태로 돌아갑니다. 토크 값이 높을 경우 사용자가 힘을 주어 밀고 있는 상황이므로 위치를 더 이동시킵니다.

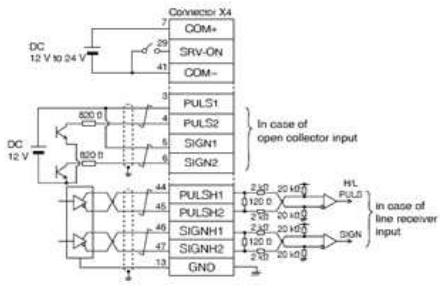
1) 토크제어(DAC)

AC서보앰프의 TRQR(14)에 입력하는 전압에 따라 모터에 인가되는 전류를 제어할 수 있습니다. 입력 전압은 MCU의 DAC(Digital-to-Analog Converter)를 이용하여 생성합니다. DC 0V~3.3V를 생성할 수 있으며, 코드에서는 8bit(0~255) 값으로 조절할 수 있습니다.



2) 위치제어(TIM_OC, Output Compare)

AC서보앰프의 PULSH(44, 45), SIGNH(46,47)에 입력하는 펄스에 따라 모터가 해당 위치로 회전합니다. 입력 펄스는 MCU의 TIM_OC(Timer Output Compare) 기능을 이용하여 생성합니다.



Command pulse format	Signal title	Positive direction command	Negative direction command
90° phase difference 2-phase pulse (A + B-phase)	PULS SIGN	A-phase B-phase t1 t1 B-phase advances to A by 90°.	t1 t1 B-phase delays from A by 90°.

타이머 설정		채널 초기 값 (카운터/전압)	
clock	168MHz	CH1	0 / LOW
prescaler	3360-1 (168MHz/3360=50KHz, 20us)	CH2	0 / HIGH
counter period	2-1 (50KHz/2=25KHz, 40us)	CH3	1 / LOW
toggle	25KHz(40us) / 주기 : 12.5KHz(80us)	CH4	1 / HIGH
pulse/ms	50/ms (12.5/ms*2ch)		
회전각	1.8° (10000:360=50:1.8)		

코드에서는 1ms당 1.8° 회전하도록 하였습니다.

```
void RotateCommand(int pos)
{
        HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_1);
        HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_2);
        HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_3);
        HAL_TIM_OC_Start(&htim1, TIM_CHANNEL_4);
        HAL_Delay(pos);
        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_1);
        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_2);
        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_3);
        HAL_TIM_OC_Stop(&htim1, TIM_CHANNEL_4);
        HAL_Delay(10-pos);
}
```

3) 토크센서

회전토크센서의 값은 RS-232(USART) 통신을 통해 MCU로 수신됩니다. 수신 인터럽트는 DMA(Direct Memory Access)방식으로 처리하여 잦은 인터럽트로 인한 무리가 발생하지 않도록 하였습니다.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
{ //RX callback function
  if(huart->Instance==USART6)
    {
      HAL_UART_Receive_DMA(&huart6, (uint8_t *)usart6_buffer, 13);
      readTorque();
    }
}
```

회전토크센서의 송신 프로토콜은 아래와 같습니다.

Byte 0	1	2	3	4	5	6	7	8	9	10	11	12
STX	ID	부호	0	x100	x10	x1	.	x0.1	x00.1	판정	ETX	NULL
BF	D6	+ : 96 - : 56	FB	0 : 0x9F 1 : 0x9D 2 : 0x9B 3 : 0x99 4 : 0x97 5 : 0x95 6 : 0x93 7 : 0x91 8 : 0x8F 9 : 0x8D			A3	0 : 0x9F 1 : 0x9D 2 : 0x9B 3 : 0x99 4 : 0x97 5 : 0x95 6 : 0x93 7 : 0x91 8 : 0x8F 9 : 0x8D		67	F9	00

MCU에서 위의 13Byte 프로토콜로 수신되는 패킷을 받아서 정수형으로 변환하여 torque 변수에 저장합니다. 토크 측정값은 소수점 2자리이므로 100을 곱하여 int형 변수에 저장합니다.

```
void readTorque()
{
  if((usart6_buffer[0]==0xbf)&&(usart6_buffer[11]==0xf9))
        {
          for(int i=1;i<=3;i++)
            {
              usart6_data[i]=(((~usart6_buffer[i+3])>>1)&0x0f);
            }
          if(usart6_buffer[7]==0xa3)
            { usart6_data[4]='.'; }
          usart6_data[5]=(((~usart6_buffer[8])>>1)&0x0f);
          usart6_data[6]=(((~usart6_buffer[9])>>1)&0x0f);

          torque = (usart6_data[1]*10000)+(usart6_data[2]*1000)+
              (usart6_data[3]*100)+(usart6_data[5]*10)+(usart6_data[6]*1);

          if(usart6_buffer[2]==0x96)
            {
              usart6_data[0]='+';
            }
          else if(usart6_buffer[2]==0x56)
            {
              usart6_data[0]='-';
              torque *= -1;
            }
        }
}
```

4. 원운동 저항운동모드

```
void Rotate_Resist(void)
{
  static uint8_t ResistTorque = 0;

  switch(Step)
  {
    case 0:
          ResistTorque = RESIST_TORQUE_DEFAULT;
        break;
    case 1:
          ResistTorque = RESIST_TORQUE_STEP1;
        break;
    case 2:
          ResistTorque = RESIST_TORQUE_STEP2;
        break;
    case 3:
          ResistTorque = RESIST_TORQUE_STEP3;
        if(!(SysStatus&MASK_MBRK))
        {
         HAL_GPIO_WritePin(GPIOG, GPIO_PIN_9, GPIO_PIN_RESET);
             SysStatus|=MASK_MBRK;
        }
        break;
    }
  HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_8B_R, ResistTorque);

  EncoderCounter = TIM2->CNT;
  pulse = (int)EncoderCounter;

  if(!(SysStatus&MASK_CL))
    {
      HAL_GPIO_WritePin(GPIOG, GPIO_PIN_1, GPIO_PIN_RESET);
      HAL_GPIO_WritePin(GPIOG, GPIO_PIN_4, GPIO_PIN_RESET);
      SysStatus|=MASK_CL;
    }
  HAL_Delay(10);
}
```

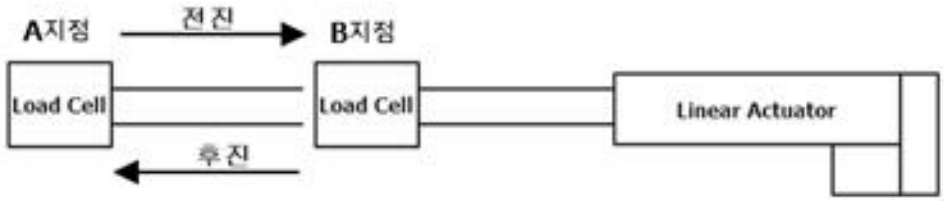
DAC(Digital-to-Analog Converter)를 이용하여 단계별 전압을 AC서보 앰프로 인가하여 모터의 인가 전류를 제어하여 부하를 생성합니다.

단계	인가전압	모터부하	브레이크부하	최종부하	토크
1	0.62V	20N	0N	20N	0.8Nm
2	0.78V	35N	0N	35N	1.4Nm
3	0.78V	35N	15N	50N	2.0Nm

필요 부하가 35N 이상일 경우 모터의 안전을 위해 마그네틱브레이크(MBRK)를 이용하여 부하를 추가 생성합니다. 위치 제어를 하지 않으면 모터가 원점으로 돌아가기 때문에 위치 편차를 초기화(CLear)하여 원점을 변경시킵니다.

5. 병진운동 Active-assisted 모드

병진운동 Active-assisted모드는 리니어 액츄에이터가 두 지점을 일정한 토크로 왕복합니다. 시작 시에는 대기하다가 사용자의 힘을 로드셀로 감지하면 A지점에서 B지점으로 이동 후 대기, B지점에서 A지점으로 이동 후 대기하는 동작을 반복합니다.



대기 지점에서 사용자가 밀거나 당기려는 힘을 감지하면 단계별 전류를 인가하여 전후진 동작합니다.

방향 / 단계	전진		후진	
	동작조건	인가전류	동작조건	인가전류
1	미는힘>0.5kg	-1.1A	당기는힘<-0.5kg	1.1A
2	미는힘>0.5kg	-1.0A	당기는힘<-0.5kg	1.0A
3	미는힘>0.5kg	-0.9A	당기는힘<-0.5kg	0.9A

```
void Translate_Assist(void)
{
  static int AssistForce = 0;
  CheckPosition(CH1);
  CheckPosition(CH2);
  switch(Step)
    {
      case 0:
            AssistForce = ASSIST_FORCE_DEFAULT;
            break;
      case 1:
            AssistForce = ASSIST_FORCE_STEP1;
           break;
      case 2:
            AssistForce = ASSIST_FORCE_STEP2;
           break;
      case 3:
            AssistForce = ASSIST_FORCE_STEP3;
         break;
    }
  if(force > AssistForce)
    {
      direction = 2;  // forward
    }
  else if(force < (AssistForce*(-1)))
    {
      direction = 1;  // backward
    }
  if(ActuatorStatus==0)
    {
            CurrentCommand(CH1, 0.0);
            CurrentCommand(CH2, 0.0);
            ActuatorStatus = 3;
    }
```

```
  else if(ActuatorStatus==1)
  {         if((stroke1>=TRANS_POINT_A)||(stroke2>=TRANS_POINT_A))
            { ActuatorStatus = 3; }
            else
            {
                switch(Step)
                {
                  case 0:
                    CurrentCommand(CH1, 1.0);
                    CurrentCommand(CH2, 1.0);
                     break;
                  case 1:
                    CurrentCommand(CH1, 1.1);
                    CurrentCommand(CH2, 1.1);
                    break;
                  case 2:
                    CurrentCommand(CH1, 1.0);
                    CurrentCommand(CH2, 1.0);
                    break;
                  case 3:
                    CurrentCommand(CH1, 0.9);
                    CurrentCommand(CH2, 0.9);
                    break;
                }
            }
  }
  else if(ActuatorStatus==2)
  {         if((stroke1<=TRANS_POINT_B)||(stroke2<=TRANS_POINT_B))
            { ActuatorStatus = 3; }
            else
            {
                switch(Step)
                {
                  case 0:
                   CurrentCommand(CH1, -1.0);
                   CurrentCommand(CH2, -1.0);
                   break;
                  case 1:
                   CurrentCommand(CH1, -1.1);
                   CurrentCommand(CH2, -1.1);
                   break;
                  case 2:
                   CurrentCommand(CH1, -1.0);
                   CurrentCommand(CH2, -1.0);
                   break;
                  case 3:
                   CurrentCommand(CH1, -0.9);
                   CurrentCommand(CH2, -0.9);
                   break;
                }
            }
  }
  else if(ActuatorStatus==3)
  {         if((stroke1<=TRANS_POINT_B)||(stroke2<=TRANS_POINT_B))
            {
                      if(direction==1)
                      { ActuatorStatus = 1; }
            }
            else if((stroke1>=TRANS_POINT_A)||(stroke2>=TRANS_POINT_A))
            {
                      if(direction==2)
                      { ActuatorStatus = 2; }
            }
            else
            {
                      CurrentCommand(CH1, 0.0);
                      CurrentCommand(CH2, 0.0);
                      ActuatorStatus = 3;
            }
  }
}
```

1) 로드셀

로드셀에 가해진 압력은 인디케이터에 전압으로 입력되어 ADC(Analog-to-Digital Converter)에 의해 Data로 변환 후 RS-232 통신으로 MCU에 10Byte 프로토콜로 전송됩니다.

Byte0	Byte1	Byte2	Byte3	Byte4	Byte5	Byte6	Byte7	Byte8	Byte9
STX	부호	x100	x10	x1	.	x0.1	x0.01	ETX	NULL
BF	+ : 96 - : 56	0 : 0xFB 1 : 0xEB 2 : 0xDB 3 : 0xCB 4 : 0xBB 5 : 0xAB 6 : 0x9B 7 : 0x8B 8 : 0x7B 9 : 0x6B			A3	0 : 0x9F 1 : 0x9D 2 : 0x9B 3 : 0x99 4 : 0x97 5 : 0x95 6 : 0x93 7 : 0x91 8 : 0x8F 9 : 0x8D		F9	00

MCU에서는 수신 프로토콜을 DMA 방식으로 수신하고, 수신된 패킷을 분석하여 변수(force)에 저장합니다.

```
void readLoadcell()
{
  if((usart2_buffer[0]==0xbf)&&(usart2_buffer[8]==0xf9))
        {
          usart2_data[1]=(((~usart2_buffer[2])>>4)&0x0f);

          for(int i=2;i<=3;i++)
            {
              if(usart2_buffer[i]==0xfb)
              {
                    usart2_data[i]=(((~usart2_buffer[i+1])>>4)&0x0f);
                    }
              else
              {
                    usart2_data[i]=(((~usart2_buffer[i+1])>>1)&0x0f);
              }
            }
          if(usart2_buffer[5]==0xa3)
            { usart2_data[4]='.'; }
          for(int j=5;j<=6;j++)
            {
              usart2_data[j]=(((~usart2_buffer[j+1])>>1)&0x0f);
            }

          force = (usart2_data[1]*10000)+(usart2_data[2]*1000)+
              (usart2_data[3]*100)+(usart2_data[5]*10)+(usart2_data[6]*1);

          if(usart2_buffer[1]==0x96)
            {
              usart2_data[0]='+';
            }
          else if(usart2_buffer[1]==0x56)
            {
              usart2_data[0]='-';
              force *= -1;
            }
        }
}
```

2) 모터드라이버

MCU와 UART to RS232통신으로 연결하여 요청 시 2채널 리니어액츄에이터의 위치값을 MCU로 송신하고, MCU로 부터 제어커맨드를 수신하여 리니어액츄에이터를 제어합니다. 통신프로토콜은 13byte Serial Binary Packet으로 value는 리틀 엔디안 방식으로 전송합니다.

1st byte	2nd byte	3rd byte	4th ~ 11th byte	12th byte	13th byte
STX (0x02)	Length (=13)	Device ID	Message	Checksum	ETX (0x03)

4th byte	5th byte	6th byte	7th byte	8th byte	9th byte	10th byte	11th byte
Command	Index		Sub-index	0	0	0	0

4th byte	5th byte	6th byte	7th byte	8th byte	9th byte	10th byte	11th byte
Command	Index		Sub-index	Value(INT8)	0	0	0
				Value(INT16)		0	0
				Value(INT32)			
				Value(FLOAT)			

MCU에서는 프로토콜에 맞추어 위치 값을 요청하는 패킷을 송신합니다.

```
void CheckPosition(uint8_t channel)
{
        usart5_protocol[0]=STX;
        usart5_protocol[1]=13;  // data length
        usart5_protocol[2]=1;
        usart5_protocol[3]=REQUEST_READ+INT32;
        usart5_protocol[4]=POSITION;
        usart5_protocol[5]=0;
        usart5_protocol[6]=channel;
        usart5_protocol[7]=0;
        usart5_protocol[8]=0;
        usart5_protocol[9]=0;
        usart5_protocol[10]=0;
        usart5_protocol[11]=CheckSum(usart5_protocol,2,10);
        usart5_protocol[12]=ETX;
        HAL_UART_Transmit(&huart5, (uint8_t*)usart5_protocol, 13, 10);
        //HAL_UART_Receive_DMA(&huart5, (uint8_t *)usart5_buffer, 13);
        HAL_Delay(10);
}
```

모터드라이버에서는 MCU에서 위치 값 요청을 받으면 프로토콜에 맞추어 위치 값을 송신합니다. MCU에서는 2채널 액츄에이터 각각의 위치 정보를 DMA(Direct Memory Access)방식으로 수신하고, 수신된 패킷을 분석하여 변수(stroke1, stroke2)에 저장합니다.

```
void readMotorDriver()
{
	int value=0;

	if((usart5_buffer[0]==STX)&&(usart5_buffer[12]==ETX))
	{

		if((usart5_buffer[3]&0xf0)==RESPONSE_READ)
		{
			value=((int)usart5_buffer[7]<<0)|
				((int)usart5_buffer[8]<<8)|
				((int)usart5_buffer[9]<<16)|
				((int)usart5_buffer[10]<<24);
			if(usart5_buffer[6]==CH1)
			{
				stroke1=value;
			}
			else if(usart5_buffer[6]==CH2)
			{
				stroke2=value;
			}
		}
		else if((usart5_buffer[3]&0xf0)==RESPONSE_WRITE)
		{

		}
	}
```

전류(토크)제어는 암페어(A) 단위로 값을 전송해야하므로 FLOAT형태로 전송합니다. FLOAT을 손실 없이 4Byte로 분할하기 위해 INT32로 변환할 수 있는 공용체 변수를 이용합니다.

```
union{
	uint32_t ui32t;
	float flt;
}u1,u2;

void CurrentCommand(uint8_t channel, float value)
{
	u1.flt=value;

	usart5_protocol[0]=STX;
	usart5_protocol[1]=13;
	usart5_protocol[2]=1;
	usart5_protocol[3]=REQUEST_WRITE+FLOAT;
	usart5_protocol[4]=CURRENT_CMD;
	usart5_protocol[5]=0;
	usart5_protocol[6]=channel;
	usart5_protocol[7]=(u1.ui32t>>0)&0xff;   //(uint8_t)(*((uint32_t*)&value)>>0);
	usart5_protocol[8]=(u1.ui32t>>8)&0xff;//(uint8_t)((*((uint32_t*)&value)>>8)&0xff);
	usart5_protocol[9]=(u1.ui32t>>16)&0xff;//(uint8_t)((*((uint32_t*)&value)>>16)&0xff);
	usart5_protocol[10]=(u1.ui32t>>24)&0xff;//(uint8_t)((*((uint32_t*)&value)>>24)&0xff);
	usart5_protocol[11]=CheckSum(usart5_protocol,2,10);
	usart5_protocol[12]=ETX;
	HAL_UART_Transmit(&huart5, (uint8_t*)usart5_protocol, 13, 10);
	HAL_Delay(10);
}
```

2채널 리니어액츄에이터를 제어하기 위해 제어 지령은 각 채널 당 10ms 간격으로 시분할하여 송신합니다.

5. 병진운동 저항운동모드

병진운동 저항운동모드는 로드셀에 가하는 반력을 감지하여 단계별 기준 반력보다 클 경우 리니어 액츄에이터를 동작시키고, 작을 경우 그 위치에서 정지합니다. 미는 힘(압축력)을 감지하면 전진, 당기는 힘(인장력)을 감지하면 후진합니다.

기준 반력 보다 큰 힘이 감지될 경우 단계별로 인가 전류를 다르게 하여 부하를 조절합니다. 인가 전류가 클수록 부하가 줄어들어 사용자 입장에서는 쉬운 이동이 가능한 반면 인가 전류가 작을수록 부하가 커져서 이동시키기 어렵습니다.

방향 / 단계	전진		후진	
	동작조건	인가전류	동작반력	인가전류
1	미는힘>3.00kg	-1.1A	당기는힘<-3.00kg	1.1A
2	미는힘>6.00kg	-1.0A	당기는힘<-6.00kg	1.0A
3	미는힘>9.00kg	-0.9A	당기는힘<-9.00kg	0.9A
정지	-	0.0A	-	0.0A

대기 상태일 때 단계별로 정해진 힘 이상의 미는 힘이 로드셀로 측정되면 리니어 액츄에이터에 전진 방향의 전류를 인가합니다. 당기는 힘이 측정되면 후진 방향의 전류를 인가합니다. 단계별 힘 보다 약한 힘이 측정될 경우 0A를 인가하여 정지합니다.

```
void Translate_Resist(void)
{
  static int ResistForce = 0;

  CheckPosition(CH1);
  CheckPosition(CH2);

  switch(Step)
  {
    case 0:
          ResistForce = RESIST_FORCE_DEFAULT;
        break;
    case 1:
          ResistForce = RESIST_FORCE_STEP1;
        break;
    case 2:
          ResistForce = RESIST_FORCE_STEP2;
        break;
    case 3:
          ResistForce = RESIST_FORCE_STEP3;
        break;
    }
```

```
    if(force > ResistForce)
    { direction = 2;  // backward }
    else if(force < (ResistForce*(-1)))
    { direction = 1;  // forward }
    else
    { direction = 0;  // stop }

    if(ActuatorStatus==0)
    {
            CurrentCommand(CH1, 0.0);
            CurrentCommand(CH2, 0.0);
            ActuatorStatus = 3;
    }
    else if(ActuatorStatus==1)
    {
        switch(Step)
        {
          case 0:
           CurrentCommand(CH1, 1.0);
           CurrentCommand(CH2, 1.0);
           break;
          case 1:
           CurrentCommand(CH1, 1.1);
           CurrentCommand(CH2, 1.1);
           break;
          case 2:
           CurrentCommand(CH1, 1.0);
           CurrentCommand(CH2, 1.0);
           break;
          case 3:
           CurrentCommand(CH1, 0.9);
           CurrentCommand(CH2, 0.9);
           break;
          }
        ActuatorStatus = 3;
    }
    else if(ActuatorStatus==2)
    {
        switch(Step)
        {
          case 0:
           CurrentCommand(CH1, -1.1);
           CurrentCommand(CH2, -1.1);
           break;
          case 1:
           CurrentCommand(CH1, -1.1);
           CurrentCommand(CH2, -1.1);
           break;
          case 2:
           CurrentCommand(CH1, -1.0);
           CurrentCommand(CH2, -1.0);
           break;
          case 3:
           CurrentCommand(CH1, -0.9);
           CurrentCommand(CH2, -0.9);
           break;
        }
        ActuatorStatus = 3;
    }
    else if(ActuatorStatus==3)
    {
            if(direction==1)
            { ActuatorStatus = 1; }
            else if(direction==2)
            { ActuatorStatus = 2; }
            else
            {
                    CurrentCommand(CH1, 0.0);
                    CurrentCommand(CH2, 0.0);
                    ActuatorStatus = 3;
            }
    }
}
```